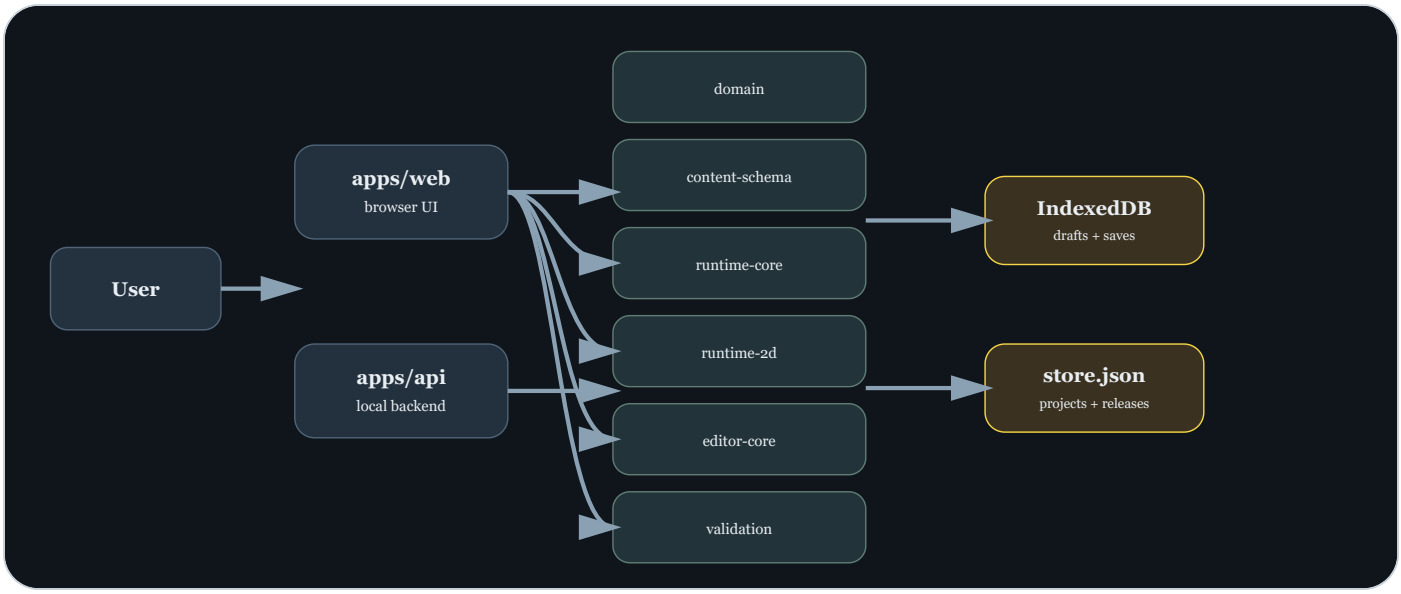
ACS MILESTONE 8+

System Reference

A technical map of how ACS fits together, with concrete end-to-end flows for runtime movement, inspection, editor tile painting, validation, persistence, and rendering.

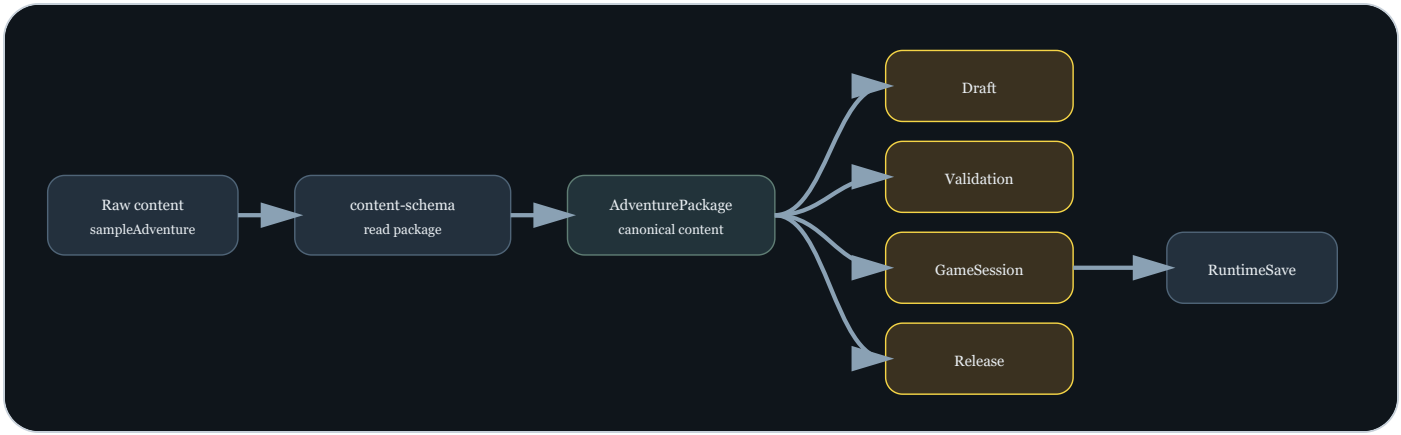
Companion documents: the user guide explains how to use the application; this system reference explains what happens internally. The Markdown source includes Mermaid diagrams, and this PDF includes readable rendered equivalents plus screenshots.

High-Level Architecture



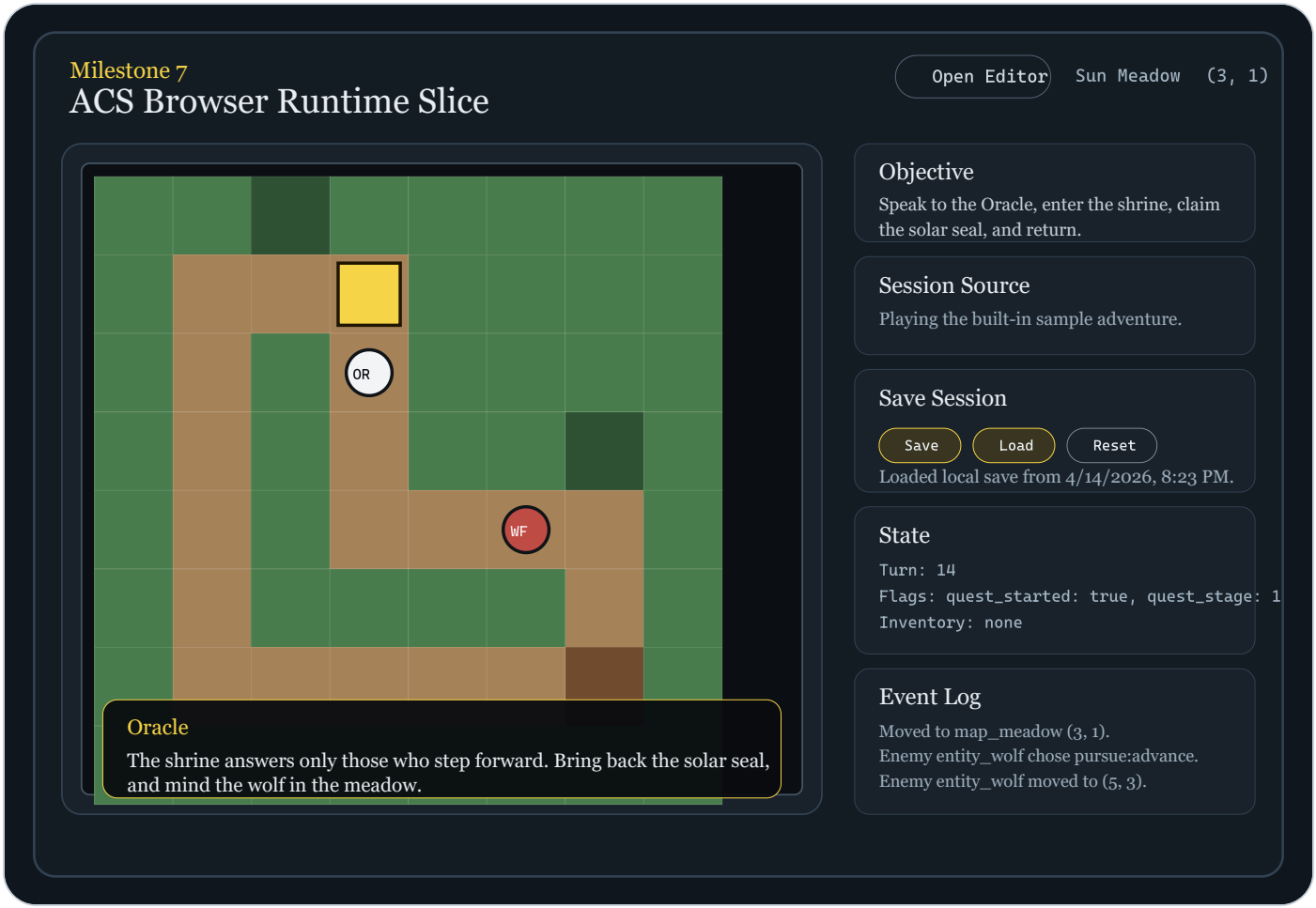
Area	Package or app	Responsibility
Shared vocabulary	`packages/domain`	IDs, maps, entities, triggers, dialogue, actions, and conditions.
Content ingestion	`packages/content-schema`	Reads raw authored content and normalizes it into an `AdventurePackage`.
Simulation	`packages/runtime-core`	Commands, state mutation, triggers, dialogue, enemy turns, snapshots, and events.
Rendering	`packages/runtime-2d`	Draws runtime state to canvas without owning game rules.
Editing	`packages/editor-core`	Pure draft operations such as `setTileAt` and `moveEntityInstance`.
Validation	`packages/validation`	Publish-readiness checks shared by browser and API.
Persistence	`packages/persistence`	IndexedDB runtime saves and local editor drafts.
Backend	`apps/api`	Projects, releases, validation endpoint, and file-backed storage.

Data Model Layers



Key distinction: a release is immutable content, while a save is runtime state. A save wraps the engine's existing `RuntimeSnapshot`, so persistence does not invent a second save-game model.

Current Screens



Milestone 8

ACS Construction Set MVP

Back To Pla | Save Draft | Reset Draft | Playtest Draft

Map: Inner Shrine
Mode: Tiles
Tile: floor

ACTIVE BRUSH
floor

stone	stone	stone	stone	stone	stone
stone	floor	floor	floor	floor	stone
stone	floor	floor	floor	altar	stone
stone	floor	floor	floor	floor	stone
stone	floor	floor	floor	floor	stone
stone	door	stone	stone	stone	stone

Painting tiles on Inner Shrine. Brush tile: floor. Click and drag to paint multiple cells.

Metadata

Title
Oracle of the Solar Seal

Description
A tiny milestone slice with movement, dialogue, triggers, enemy behavior, and map travel.

Project & Release

Connected to local API as Local Designer.
Project Oracle of the Solar Seal (proj_0001)
Server validation passed with 0 warnings.

Validate Draft | Create Project | Publish | Save Project | Open Latest

v2 published 4/14/2026 8:31 PM - 0 errors, 0 warnings

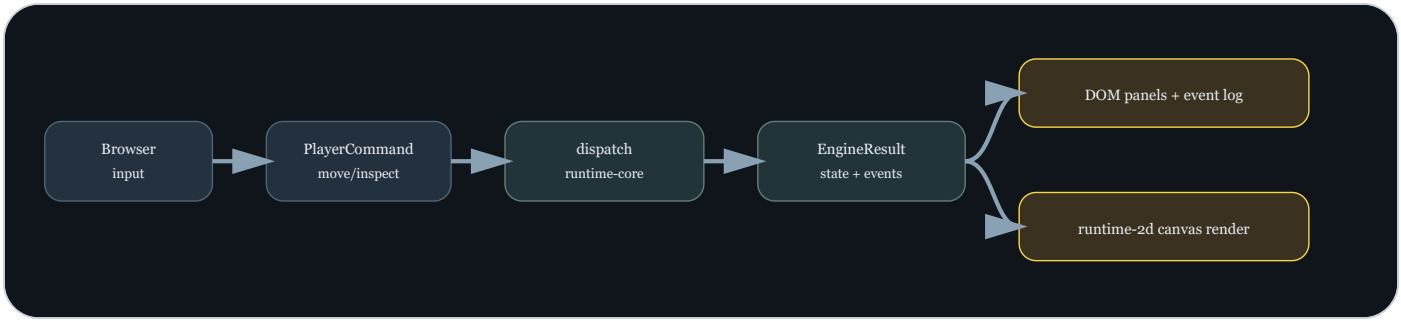
Validation

Draft is publishable with 0 warnings.
No validation issues.

Entities On Map

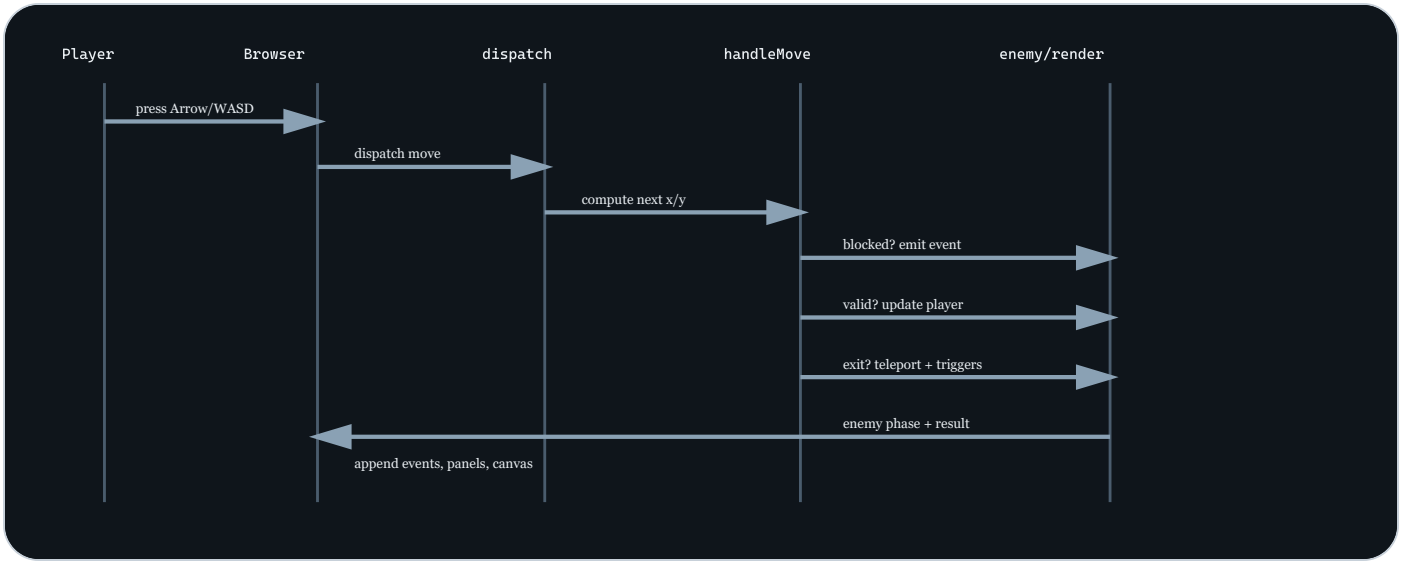
No entities on this map.

Runtime Input To Rendering



Runtime input follows a narrow path: `apps/web/src/index.ts` turns browser input into a `PlayerCommand`, `runtime-core` mutates session state and emits events, then the browser updates panels and asks `runtime-2d` to redraw.

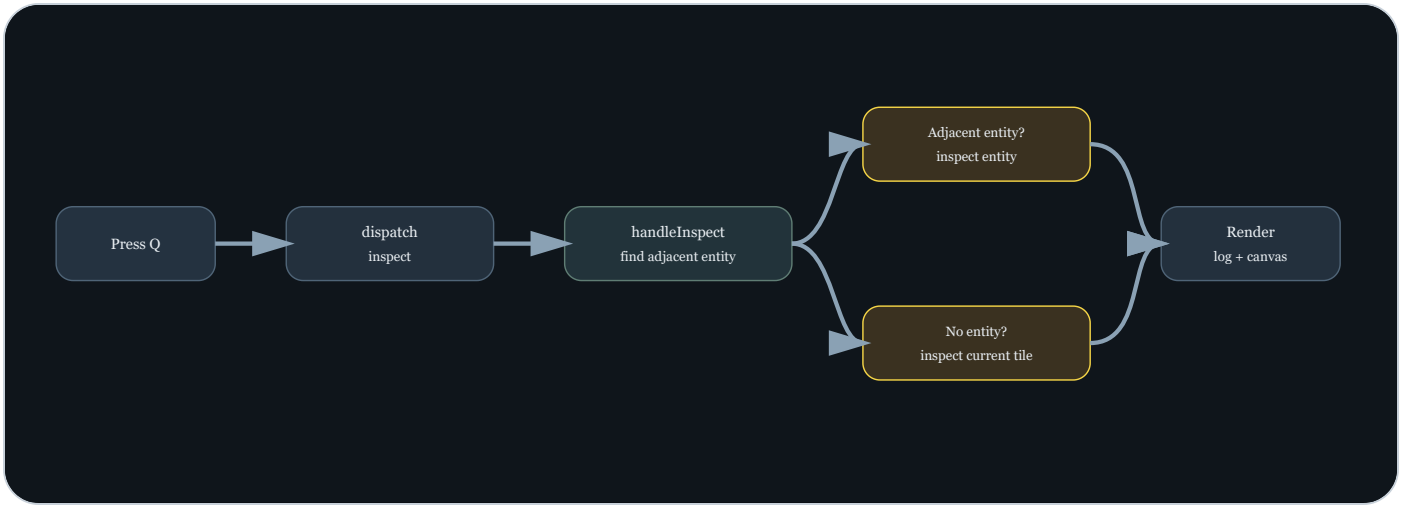
Use Case 1: Move Command



1. The keydown handler in `apps/web/src/index.ts` maps Arrow keys and `WASD` to `{ type: "move", direction }`.
2. `runCommand` calls `session.dispatch(...)`, appends returned events, and calls `renderEverything`.
3. `dispatch` calls `handleMove` and marks the command as enemy-phase eligible.
4. `handleMove` computes the next coordinate, rejects bounds or occupied destinations, or updates `state.player`.
5. If the destination is an exit, the engine changes `currentMapId`, teleports the player, and runs map-load and tile triggers.
6. The enemy phase runs after the move command, then the browser receives `EngineResult(state, events)`.
7. The browser updates map name, player position, flags, inventory, dialogue, event log, and canvas.

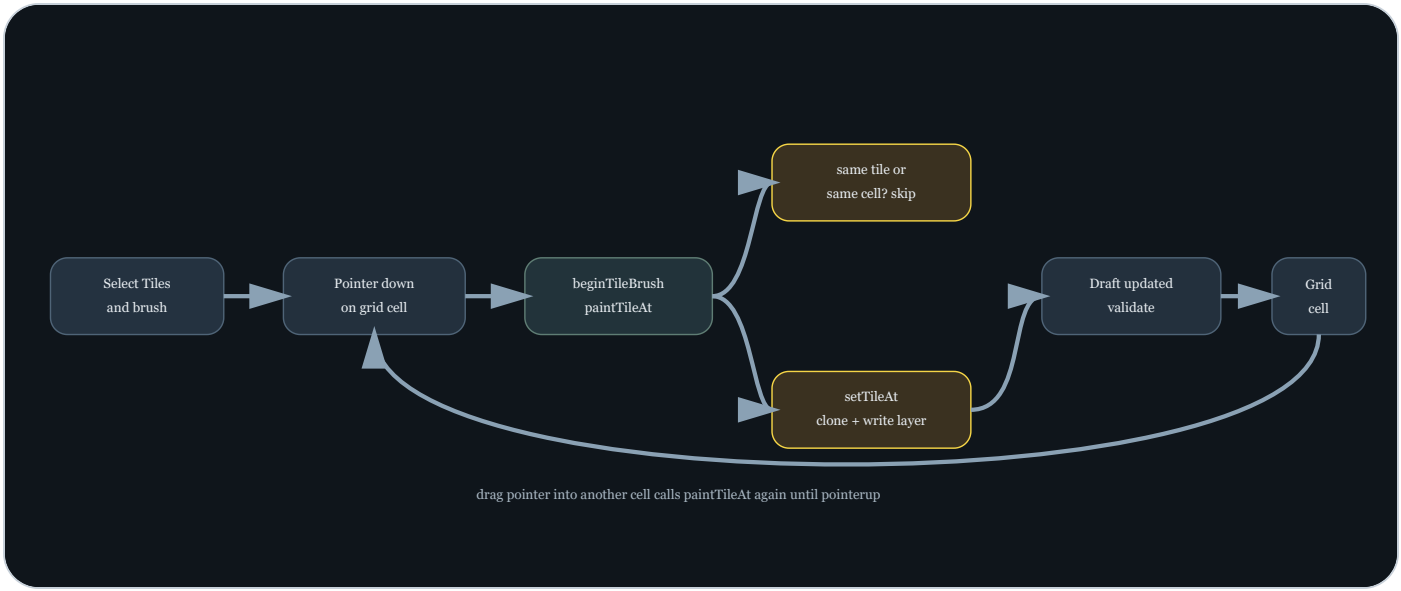
Current behavior: even a blocked move currently lets the enemy phase run, because `dispatch` marks all move commands as turn-resolving before knowing whether the move succeeds.

Use Case 2: Inspect Command



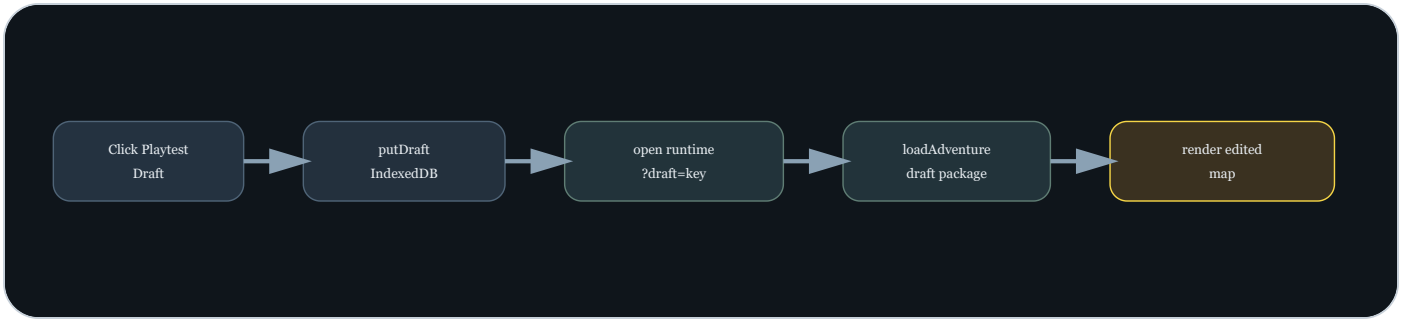
1. The browser maps `Q` to `{ type: "inspect" }`.
2. `dispatch` calls `handleInspect(undefined, events)` and marks inspect as enemy-phase eligible.
3. With no explicit direction, `findEntityInDirection` searches for the first active adjacent entity on the current map.
4. If an adjacent entity exists, the engine emits `inspectResult` for that entity.
5. If no adjacent entity exists, the engine emits an `inspectResult` describing the player's current coordinate and map id.
6. Inspect does not directly move the player, grant items, set flags, or start dialogue.
7. The enemy phase currently runs afterward, so the canvas may change if an enemy moves.

Use Case 3: Editor Tile Brush



1. The designer selects `Tiles` mode and picks a tile id. The browser stores it in `selectedTileId`.
2. `renderBrushPreview` updates the visible active brush.
3. Each grid cell has a `pointerdown` handler in tile mode. Pressing a cell calls `beginTileBrush(x, y)`.
4. `beginTileBrush` sets `isTileBrushActive = true`, clears the duplicate-cell guard, and calls `paintTileAt`.
5. `paintTileAt` skips if the editor is not in tile mode, if the same cell was already painted in this drag, or if the cell already has the selected tile.
6. If the tile differs, the browser calls `setTileAt(draft, currentMapId, x, y, tileId)` from `editor-core`.
7. `setTileAt` clones the package, finds the map and active layer, calculates `index = y * map.width + x`, writes the tile id, and returns a new draft.
8. The browser refreshes the changed grid cell, reruns local validation, and updates project/publish controls.
9. Dragging into another cell calls `paintTileAt` through `pointerenter`. Pointer up ends the brush.

Editor To Playtest



Painting a tile changes only the editor draft. To see it in play, the editor saves the draft to IndexedDB and opens the runtime with a draft query parameter. The runtime loads that same `AdventurePackage`, so there is no separate editor-only map format.

Validation, Publishing, Save, And Load

Validation and publishing

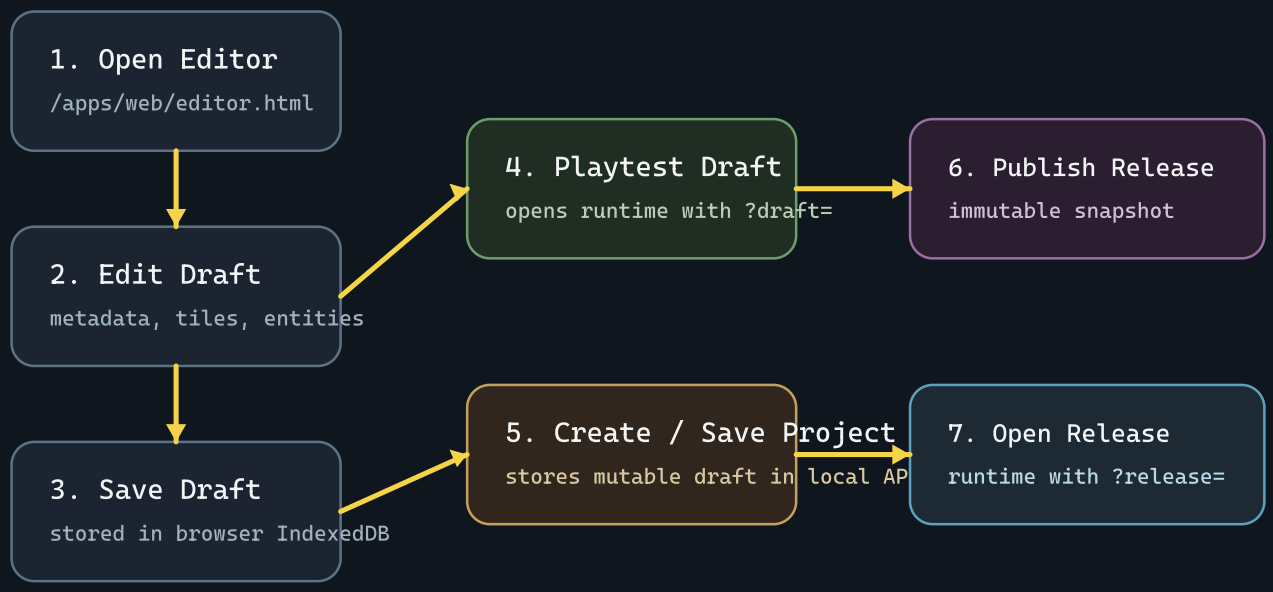
1. Local edits rerun `validateAdventure(draft)`.
2. `Validate Draft` calls the API validation endpoint.
3. Project save and publish are disabled when local validation is blocking.
4. The API validates again before creating an immutable release.

Runtime save/load

1. `Save` calls `session.serializeSnapshot()`.
2. `persistence.saveSession` stores the snapshot in IndexedDB.
3. `Load` retrieves the record and calls `engine.loadAdventure(activeAdventure, snapshot)`.
4. The browser rerenders from restored state.

ACS User Workflow

How the current application is used from editing to playtesting to publishing.



Extension Points

- A higher-resolution renderer can live beside `runtime-2d` as long as it consumes `AdventurePackage` and `GameSessionState`.
- A future 3D renderer should not change browser input or editor logic unless the content model needs richer spatial metadata.
- Real-time play would most likely evolve the command scheduling and enemy phase in `runtime-core`.
- Richer enemy AI belongs in `runtime-core` or a future AI package, not in the DOM or canvas renderer.
- New editor creation tools should start as pure functions in `editor-core`, then be wired into `apps/web/src/editor.ts`.